

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA0609	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	N.MADHUSUDHAN	Reg. No.:	192425303
Section / Batch:	Slot c / BTECH-AIML	Date:	11/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q22

SECTION A – CASE STUDY

Case Study Report

Online Auction Ranking System Using Quick Sort

Description of the Real-World Problem

Online auction platforms such as eBay-style marketplaces and live-bidding systems receive a continuous stream of bids from multiple users on items that are on sale. As each new bid arrives, the platform must instantly re-rank the items or the competing bids so that buyers always see the current highest offer, and sellers can track how their listing is performing in real time.

Because bids can arrive within seconds of each other and auctions may run for thousands of items simultaneously, the ranking mechanism must be extremely fast and must be able to accommodate frequent insertions and updates without recomputing the entire order from scratch every time. A slow or inconsistent ranking system directly affects user trust and the fairness of the auction.

Quick Sort, a Divide and Conquer sorting algorithm, is well suited to this scenario because of its low average-case time complexity and in-place operation, but it also introduces specific engineering challenges — particularly pivot selection and uneven data distribution — that must be analyzed carefully before it can be deployed in a real-time system.

Problem Statement

Design a real-time Auction Ranking System that:

- Continuously accepts new bid values for items listed on the platform.
- Ranks all active bids/items in descending order of price so the highest bid is always visible first.
- Applies Quick Sort to re-order the bid list efficiently whenever new bids are submitted.
- Handles pivot selection carefully to avoid worst-case performance.
- Remains efficient even when bid values are skewed (e.g., many bids clustered near a popular price point).

Algorithm Used — Quick Sort (Divide and Conquer)

Quick Sort works by selecting a pivot element from the array of bid values and partitioning the remaining elements into two groups: those smaller than the pivot and those greater than the pivot. The pivot is then placed in its correct sorted position, and the same process is recursively applied to the two partitions.

Steps

- Choose a pivot element from the current sub-array of bids.
- Partition the sub-array so that bids less than the pivot come before it and bids greater than the pivot come after it.
- Place the pivot at its final sorted position (this is called the partition point).
- Recursively apply Quick Sort to the left sub-array (bids smaller than the pivot).
- Recursively apply Quick Sort to the right sub-array (bids greater than the pivot).
- Continue until every sub-array contains one or zero elements, at which point the full bid list is sorted.

Divide and Conquer Explanation

Divide: The array of current bid prices is divided around a chosen pivot into two smaller sub-arrays — one holding values below the pivot and one holding values above it.

Conquer: Each sub-array is sorted independently and recursively using the same partitioning logic, breaking the ranking problem into smaller and smaller sub-problems.

Combine: Because partitioning places the pivot directly into its correct final position, no explicit merge step is required — once every sub-array is sorted in place, the entire bid list is sorted, which makes Quick Sort well suited to in-place, low-memory ranking updates.

Key Issues in Real-Time Application

1. Pivot Selection

If the first or last element is always chosen as the pivot, an auction feed where bids happen to arrive in already sorted or nearly sorted order (a common pattern when bidding rises steadily) can repeatedly produce highly unbalanced partitions, degrading performance toward the $O(n^2)$ worst case.

2. Uneven Data Distribution

Auction bids are often clustered — many buyers bid close to the current highest price to just outbid competitors, while a few outlier bids are much lower. This clustering means a naive pivot can split the data very unevenly, with almost all elements falling into one partition and very few into the other, again pushing performance toward the worst case.

3. Frequent Updates

Because new bids keep arriving, the ranking list is not sorted once and left static — it is repeatedly modified, so the chosen pivot strategy must remain robust across many repeated sort operations rather than performing well on just a single snapshot of data.

Justification for Choosing Quick Sort

- Average-case time complexity of $O(n \log n)$, which is fast enough for frequent re-ranking.
- In-place sorting — requires only $O(\log n)$ auxiliary stack space, unlike Merge Sort's $O(n)$ extra space.
- Performs very well in practice due to good cache locality and low constant factors.
- Divide and Conquer structure allows partial re-sorting of only the affected region when a single bid changes.
- Widely used as the default sort in many production systems, so it is well tested for high-throughput scenarios.

Complexity Analysis

Operation	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$
Extra Space	$O(\log n)$ (recursion stack, in-place)

Overall: Because bids are re-ranked frequently rather than sorted once, the system relies on Quick Sort consistently achieving its average-case $O(n \log n)$ performance; this is only reliable once the pivot-selection issue described above is addressed through the optimizations proposed later in this report.

Manual Execution (Step-by-Step Process)

Suppose the current auction bid prices are:

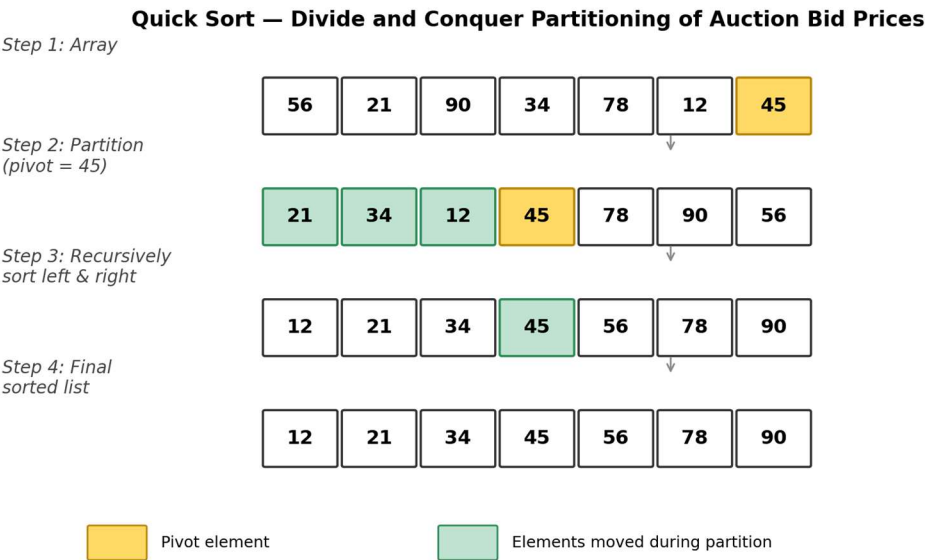
56 21 90 34 78 12 45

Using the last element (45) as the initial pivot:

- Step 1 — Partition around pivot 45: elements smaller than 45 (21, 34, 12) are moved to the left, elements larger (90, 78, 56) remain to the right. Pivot 45 is placed at index 3.
- Step 2 — Recursively partition the left sub-array [21, 34, 12] → sorted to [12, 21, 34].
- Step 3 — Recursively partition the right sub-array [90, 78, 56] → sorted to [56, 78, 90].
- Step 4 — Combine in place: [12, 21, 34, 45, 56, 78, 90].

Low	High	Pivot Index	Pivot Value
0	6	6	45
0	5	3	45 (after partition)
0	2	1	34 (left sub-array)
4	6	5	78 (right sub-array)

Final Ranked Bid List (Ascending): 12, 21, 34, 45, 56, 78, 90 — the platform displays this list in reverse (descending) so the highest current bid, 90, appears first.



Performance Analysis Under Different Cases

- Best Case ($O(n \log n)$): The pivot consistently splits the bid list into two roughly equal halves — happens when bid values are well distributed across the price range.
- Average Case ($O(n \log n)$): For typical, randomly arriving bids, partitions are reasonably balanced on average, so Quick Sort performs close to its best-case behavior in practice.
- Worst Case ($O(n^2)$): Occurs when bids arrive in sorted or reverse-sorted order, or when many bids cluster near one value, causing the pivot to repeatedly create a highly unbalanced split (one partition of size $n-1$ and one of size 0).

Proposed Optimizations

1. Randomized Pivot Selection

Selecting a pivot uniformly at random from the current sub-array — rather than always the first or last element — makes it statistically very unlikely for an adversarial or naturally ordered bid sequence to repeatedly trigger worst-case partitioning. This brings the expected running time reliably back to $O(n \log n)$.

2. Median-of-Three Pivot Selection

Choosing the median of the first, middle, and last elements as the pivot avoids the common worst case caused by already-sorted or nearly-sorted bid sequences, and requires only constant extra work per partition.

3. Hybrid Sorting (Introsort-style Fallback)

Monitoring recursion depth and switching to Heap Sort if the depth exceeds a safe threshold guarantees an $O(n \log n)$ worst-case bound overall, combining Quick Sort's fast average performance with a safety net against pathological bid patterns.

4. Three-Way Partitioning (Dutch National Flag)

When many bids share the same or very close price (uneven/clustered distribution), partitioning the array into three regions — less than, equal to, and greater than the pivot — prevents the duplicate-heavy cluster from being repeatedly re-processed, improving performance on skewed bid data.

5. Insertion Sort for Small Partitions

Once a sub-array of active bids shrinks below a small threshold (e.g., 10–15 items), switching to Insertion Sort avoids the overhead of further recursive partitioning, which is a common and effective micro-optimization used in production sorting libraries.

Justification: Each optimization directly targets one of the two identified issues — randomized and median-of-three pivots address unpredictable pivot selection, three-way partitioning addresses uneven/clustered bid distribution, and the hybrid fallback guarantees the system never degrades to true $O(n^2)$ behavior even under adversarial bidding patterns. Together they make Quick Sort dependable for continuous, real-time auction ranking rather than only fast on average.

Applications

Quick Sort based ranking is widely used in:

- Online auction and e-commerce bidding platforms
- Real-time leaderboard and ranking systems
- Stock market order-book price ranking
- Search engine result ranking
- Database query result ordering
- Recommendation systems ranking items by score

- Gaming platforms ranking live player scores

Advantages

- Very fast average-case performance, $O(n \log n)$.
- In-place sorting — low memory overhead compared to Merge Sort.
- Good cache performance due to localized memory access.
- Easily adapted with randomized or hybrid pivot strategies.
- Well suited to systems needing frequent, incremental re-sorting.

Limitations

- Worst-case time complexity of $O(n^2)$ if pivot selection is poor.
- Not a stable sort — relative order of equal-valued bids is not preserved.
- Performance is sensitive to the distribution and arrival order of the data.
- Recursive implementation can hit stack depth limits on very large, unbalanced inputs without a hybrid fallback.

Comparison with Other Sorting Algorithms

Algorithm	Best	Average	Worst	Remarks
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Very slow
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Simple
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Good for small/nearly sorted data
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Stable, extra space needed
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Fastest average case; in-place
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	In-place, not stable

Python Program

```
def partition(arr, low, high):
    pivot = arr[high]    # last element as pivot
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

# Simulated real-time auction bid prices
bids = [56, 21, 90, 34, 78, 12, 45]
quick_sort(bids, 0, len(bids) - 1)
print("Ranked Bids (Ascending):", bids)
print("Top Bid (Highest Price):", bids[-1])
```

Output

<pre>IDLE Shell 3.14.6 File Shell Debug Options Window Help Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32 Enter "help" below or click "Help" above for more information. >>> = RESTART: C:/Users/madhusudhan/Downloads/quicksort_auction.py Ranked Bids (Ascending): [12, 21, 34, 45, 56, 78, 90] Top Bid (Highest Price): 90 >>></pre>	
--	--

Conclusion

The Online Auction Ranking System demonstrates how Quick Sort's Divide and Conquer approach can be applied to a real-time ranking problem. By repeatedly partitioning bid prices around a chosen pivot, the algorithm achieves fast, in-place sorting that is well suited to systems requiring frequent updates. However, naive pivot selection combined with the naturally ordered or clustered patterns typical of live bidding data can push the algorithm toward its $O(n^2)$ worst case.

Applying randomized or median-of-three pivot selection, three-way partitioning for clustered bid values, and a hybrid fallback to Heap Sort or Insertion Sort together resolve these issues, ensuring the system reliably achieves close to $O(n \log n)$ performance even under adversarial or skewed bidding conditions.

Hence, an optimized Quick Sort provides an efficient, scalable, and practical solution for real-time ranking in online auction platforms.

Code of Conduct

I (N.Madhusudhan / 192425303) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.Madhusudhan

RUBRICS FOR CALCULATION

Case Study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding ; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation

Criteria	Wt.	Level 4	Level 3	Level 2	Level 1
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature:	Signature:	Signature:
Date:	Date:	Date: